DNS-Rebinding / TOCTOU SSRF

DNS-Rebinding / TOCTOU SSRF — Gap Analysis + Resolved-IP Recheck Design

Revision: 1 **Last modified:** 2026-07-01T17:18:18Z **Status:** Design + autonomous gap-demonstration. The static SSRF floor (Squid `http_access` host/dest ACL + Dante first-match `socks pass/block` ACL) does NOT cover the DNS-rebinding / TOCTOU sub-class of SSRF. This document (a) proves the gap in policy logic autonomously via the local-stub test, and (b) designs the fix — a post-DNS resolved-IP recheck (Stripe smokescreen, the survey's #1 recommendation). INCORPORATING smokescreen onto the live egress path is operator-gated (§11.4.122); NOTHING here mutates the data-plane. **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. §11.4.45 integration-status-class design doc for the proxy security workstream; findings presented as FACTs from a static review + a modelled local-stub proof (§11.4.6), deductions the sources do not directly state marked **INFERENCE**. **Feature workstream:** `feature/vpn-aware-dynamic-routing` (§11.4.167) **Companion:** survey `../../research/vpn_lan_opensource_survey_20260701/SURVEY.md` §4 (smokescreen = rec #1) · proxy security `Status.md` findings #8/#9 (the Dante SSRF floor this doc extends) · local proof `../../../tests/security/dns_rebinding_ssrf_gap.sh` · sibling carve-out teeth `../../../tests/vpn_lan/ssrf_carveout_teeth.sh` · the audited floor `config/dante/sockd.conf` + `config/squid/squid.conf` (READ-ONLY).

---

1. What the DNS-rebinding / TOCTOU SSRF class is (FACT, §11.4.6)

Server-Side Request Forgery (SSRF) is the class where an attacker steers a server (here: our forward/egress proxy) into making a request to a destination the attacker chooses — most

dangerously an **internal** destination the proxy can reach but the attacker cannot: cloud-metadata (`169.254.169.254`), loopback (`127.0.0.1`), or RFC1918 private hosts (`10.x`, `172.16–31.x`, `192.168.x`).

The **DNS-rebinding / TOCTOU** sub-class defeats an allowlist that validates the **requested destination** (the hostname, or the IP it resolves to *at the moment the ACL is evaluated*) but does **not** re-validate the **actual IP the proxy finally connects to**. It exploits the time gap between *check* and *use* (TOCTOU = Time-Of-Check to Time-Of-Use):

Phase	Event	What the name resolves to	What the ACL sees
Check	proxy evaluates its allowlist for `rebind.evil.example`	a **public** IP (e.g. `1.2.3.4`)	"host allowed / IP public → PERMIT"
Use	proxy opens the connection moments later	attacker's short-TTL DNS now returns an **internal** IP (`169.254.169.254`)	*not re-checked* — proxy dials the internal IP

The attacker controls the authoritative DNS for their own hostname with a very short TTL, so the record flips between the check and the connect. The result is a connection to an internal address that the static allowlist believed it had vetted. This is precisely the class the OWASP SSRF prevention guidance calls out ("resolve the hostname and validate the resulting IP address" — because a name-or-check-time-IP allowlist is insufficient) and the "DNS rebinding" attack literature documents.

---

2. Why our static host/dest ACL misses it (FACT, §11.4.6 — the gap)

Our current SSRF floor is two static, first-match, **destination-matching** ACL sets — neither re-resolves + re-validates the connect-time IP:

- **Dante SOCKS** (`config/dante/sockd.conf`, this-doc-cited lines 29–56): ordered `socks block { … to: <cidr> }` rules for `127.0.0.0/8`, `169.254.0.0/16`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, then a final `socks pass { to: 0.0.0.0/0 command: connect }`. First-match-wins on the **requested destination**. This is a strong floor for a *directly-specified* internal destination — it is exactly what `ssrf_carveout_teeth.sh` proves

— but the block match is evaluated against the destination as presented, at rule-eval time; it is not a post-resolution re-check of the socket's final peer IP.
- **Squid HTTP-CONNECT** (`config/squid/squid.conf`, lines 22–55): `acl localnet`
  - `http_access` chain matches on the request's host/dest. For a CONNECT to a hostname, Squid's allow/deny decision keys on the requested authority, not on a re-validation of the IP the outbound socket ultimately connects to after DNS.

The gap (FACT): both floors match on the **requested host/dest**. A hostname that resolves to a public IP at ACL-check time and then re-resolves to an internal IP at connect time is not caught, because **neither floor re-checks the actual connect-time resolved IP against the internal/metadata deny ranges**. This is the survey's §4 finding: *"smokescreen adds the one thing Squid ACLs miss — post-DNS IP re-check that defeats DNS-rebinding / TOCTOU."*

Honest boundary (§11.4.6): this is a gap in **policy logic**, demonstrated on a modelled two-phase resolve. It is **NOT** a claim that helix_proxy is currently being exploited — no real rebinding request has been issued against the live proxy, and the exposure is bounded by the fact that the egress carve-out for the `10/8` VPN subnet is itself operator-gated + not yet opened. The point is structural: *were the carve-out opened with the current match-on-host/dest logic, the rebinding class would not be covered.*

2.1 Autonomous proof of the gap (local-stub, no live proxy)

`tests/security/dns_rebinding_ssrf_gap.sh` demonstrates the gap deterministically **without attacking anything** — it exercises the DECISION LOGIC of two egress policies (in the exact anti-bluff style of `ssrf_carveout_teeth.sh`) against a modelled two-phase DNS resolve. It never opens a socket, never resolves a real name, never touches the data-plane. Two policies (illustrative pseudocode):

```
# models our CURRENT static floor — decision on requested host/
dest ONLY
policy_hostname_only(host_allowed, connect_ip):
    return PERMIT if host_allowed else DENY   # connect_ip never
re-checked

# models smokescreen — host gate AND post-DNS re-validation of the
ACTUAL peer IP
policy_resolved_recheck(host_allowed, connect_ip):
    if not host_allowed:            return DENY
    if is_internal(connect_ip):     return DENY   # deny-floor re-
```

```
check
    return PERMIT
```

Modelled scenarios (a name resolving public at check time, internal at connect time) and the captured verdicts:

Scenario	check-time IP	connect-time IP	`hostname_only`	`resolved_recheck`
benign public (control)	`93.184.216.34`	`93.184.216.34`	permit	**permit** (no over-block)
rebind → metadata	`1.2.3.4`	`169.254.169.254`	**permit (GAP)**	**deny (FIX)**
rebind → loopback	`1.2.3.4`	`127.0.0.1`	**permit (GAP)**	**deny (FIX)**
rebind → RFC1918 `10.x`	`1.2.3.4`	`10.6.100.221`	**permit (GAP)**	**deny (FIX)**
disallowed host (control)	`5.6.7.8`	`5.6.7.8`	deny	deny

- **T1 (RED evidence the gap is real):** under `hostname_only`, every rebinding target is **permitted** — the proxy would then dial the internal connect-time IP. Evidence: `qa-results/security/dns_rebinding/<ts>/t1_gap_hostname_only.evidence`.
- **T2 (the fix works):** under `resolved_recheck`, every rebinding target is **denied**, while the benign stable-public host is still permitted (no over-block). Evidence: `…/t2_fix_resolved_recheck.evidence`.
- **§1.1 paired mutation (`REBIND_MUT=1`):** weakens the recheck to re-validate the *check-time* IP (the wrong, public one) instead of the connect-time IP — the exact TOCTOU regression — and asserts the fix teeth then **FAIL** (exit 1). A teeth test whose fix assertion passes regardless of the recheck would be a bluff gate (§11.4.107(10)); this proves the T2 teeth genuinely depend on the connect-time re-check. Evidence: `…/mutation.evidence`.

The check-time IP for the rebinding scenarios is deliberately **public** (`1.2.3.4`): this shows that even a static ACL that *did* validate the check-time resolved IP would still permit the rebinding target — **only re-checking the actual connect-time IP catches it**. (**INFERENCE**, §11.4.6: our floor matches on host/dest and does not perform even the weaker check-time-IP validation, so it is a superset of the vulnerable behaviour the test models.)

---

3. How Stripe smokescreen closes it (FACT, §11.4.150 multi-angle)

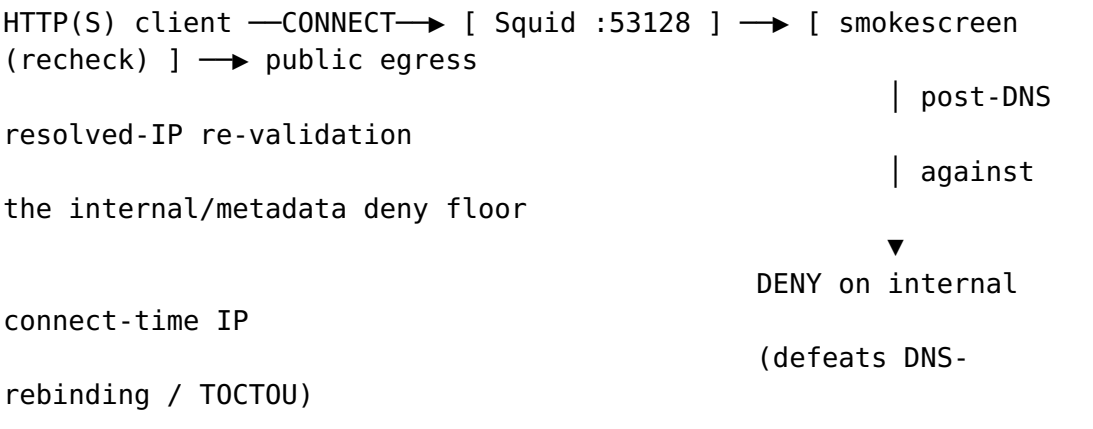
smokescreen (Stripe, MIT, Go) is an HTTP-`CONNECT` egress proxy purpose-built for this class. Per its README (source cited below), for every outbound request it: (1) checks the requested host against an **allowlist / denylist** with `open` / `report` / `enforce` modes; (2) **resolves the hostname and re-validates the resulting IP** against a configurable set of internal/deny ranges *after* DNS, blocking non-routable / private / link-local / metadata IPs; and (3) re-checks the IP the connection actually lands on — closing the TOCTOU window a static host/dest allowlist leaves open. That post-DNS resolved-IP recheck is the exact mechanism our floor lacks (§2), and it is the property the OWASP SSRF prevention guidance prescribes ("validate the resolved IP, not just the hostname"). The internal ranges are the standard non-routable blocks — RFC 1918 private space and RFC 3927 link-local (`169.254.0.0/16`, which contains the `169.254.169.254` cloud-metadata endpoint).

Deny-floor parity (FACT): the deny-floor the design's recheck re-validates against — `0.0.0.0/8`, `10.0.0.0/8`, `100.64.0.0/10`, `127.0.0.0/8`, `169.254.0.0/16`, `172.16.0.0/12`, `192.168.0.0/16` — mirrors the shipped Dante floor (`config/dante/sockd.conf`) plus this-host + CGNAT, so the recheck is a **post-DNS re-application of the same ranges we already block statically**, not a new policy surface to reason about.

---

4. Integration seam for helix_proxy (design, §11.4.6)

The recheck belongs on the **HTTP(S)-CONNECT egress path** only — smokescreen is an HTTP-CONNECT proxy and covers HTTP(S) egress; it is not an L3/SOCKS router.



```
WebDAV /
Cast-control / DIAL / ADB)  ──► [ Dante :51080 SOCKS first-match
ACL + host firewall ]
                                                  (unchanged —
smokescreen is HTTP-only)
```

- **Placement (INFERENCE, §11.4.6 — to be pinned at prototype):** smokescreen sits on the HTTP-CONNECT egress carve-out, either **behind** Squid (Squid forwards upstream through smokescreen via `cache_peer`, so the resolved-IP recheck is the last hop before egress) or **beside** Squid (HTTP(S) egress is pointed at smokescreen directly for the `10/8` carve-out). Exactly which of the two is the cleanest fit for our `never_direct` / dynamic-vs-static Squid profile is a prototype-time decision, not asserted here.
- **Config injection (§11.4.28):** the allowlist + deny-floor are injected from the project's `HELIX_BRIDGE_SUBNET` / floor config at boot, never hardcoded into the image — decoupled + reusable.
- **Rollout safety:** smokescreen's `report` mode logs what *would* be denied without enforcing, so the carve-out can be validated in report mode first, then flipped to `enforce` — mirroring the RED→GREEN discipline the proxy security Status doc already uses.
- **Honest boundary (§11.4.6):** smokescreen protects **only** the HTTP(S) CONNECT path. The L3-routed protocols (SMB/NFS/FTP/mail/ADB/Cast-control) are not HTTP and stay governed by the Dante first-match ACL + a host-level firewall (nftables) — smokescreen is a **complement to** the floor, never a replacement for it.

---

5. Catalogue-check (§11.4.74) — reuse, not reimplement

- **Ownership:** smokescreen is **not** in our orgs (`vasic-digital` / `HelixDevelopment`) — it is external (Stripe, MIT). Per §11.4.74 the verdict is **external reuse**, not a git submodule of third-party code.
- **How consumed:** as a **deployed rootless-Podman daemon** booted via the containers submodule (§11.4.76, rootless §11.4.161), config-injected + decoupled (§11.4.28) — the same pattern the survey prescribes for every external tool. No new git dependency is added to helix_proxy; no new Git remote.
- **Reuse vs extend (INFERENCE, §11.4.6):** smokescreen covers the required post-DNS resolved-IP recheck out of the box, so this is a **reuse** (≥80% fit), not an extend-upstream — pending the prototype confirming its allowlist/deny config models our `HELIX_BRIDGE_SUBNET` carve-out exactly.

---

6. Honest boundary + scope of this deliverable (§11.4.6 / §11.4.122)

This document is a **DESIGN + a gap-demonstration**, not an incorporation:

1. **The gap + the fix logic are proven autonomously NOW** — `dns_rebinding_ssrf_gap.sh` demonstrates, with captured evidence and a paired §1.1 mutation, that a host/dest-only policy misses the rebinding class and a resolved-IP recheck closes it. No live VPN, no live proxy, no real SSRF, no data-plane mutation.
2. **Actually deploying smokescreen is operator-gated (§11.4.122).** Standing up the daemon on the HTTP-CONNECT egress path, and opening the `10/8` carve-out it would guard, changes what the running System exposes — that is an operator keep-vs-connectivity decision (mirrors Status.md findings #7/#8), surfaced via §11.4.66, never adopted silently. Opening the carve-out before the recheck is in place would itself be an SSRF regression (§11.4.101 / §11.4.133), so the ordering is: recheck design (this doc) → operator approval → deploy recheck + open carve-out together.
3. **No claim of current exploitation.** The exposure is structural and conditional on the carve-out being opened with match-on-host/dest logic; the local-stub proves the *logic*, and this doc records the fix so the carve-out is never opened without it.

Next step (per survey §7 step 1): prototype smokescreen in `report` mode on a local-stub HTTP-CONNECT target that re-resolves to an internal IP, confirm `enforce` denies it, and ship a paired §1.1 mutation (an out-of-allowlist resolved IP still denies) — the autonomous slice — before proposing the operator-gated live deployment.

Sources verified 2026-07-01

- **smokescreen** (Stripe, MIT, Go — HTTP-CONNECT egress proxy; hostname allow/report/enforce + **post-DNS resolved-IP re-check** against internal/deny ranges): https://github.com/stripe/smokescreen · README: https://github.com/stripe/smokescreen/blob/master/README.md
- **Practical smokescreen — sanitizing outbound web requests** (Fly.io — the DNS-rebinding / post-resolution IP re-check rationale in operational detail): https://fly.io/blog/practical-smokescreen-sanitizing-your-outbound-web-requests/
- **OWASP — Server-Side Request Forgery Prevention Cheat Sheet** (validate the **resolved IP**, not just the hostname; deny internal/metadata ranges): https://cheatsheetseries.owasp.org/

cheatsheets/
Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- **SSRF attacks + DNS rebinding background** (the check-vs-use / TOCTOU mechanism): https://goteleport.com/blog/ssrf-attacks/
- **RFC 1918 — Address Allocation for Private Internets** (the private ranges `10/8`, `172.16/12`, `192.168/16` the recheck denies): https://www.rfc-editor.org/rfc/rfc1918
- **RFC 3927 — Dynamic Configuration of IPv4 Link-Local Addresses** (`169.254.0.0/16`, which contains the `169.254.169.254` cloud-metadata endpoint): https://www.rfc-editor.org/rfc/rfc3927
- Companion survey (rec #1 = smokescreen): `docs/research/vpn_lan_opensource_survey_20260701/SURVEY.md` §4 + §7

*Access date for all sources: 2026-07-01 (§11.4.99). FACT items (smokescreen's post-DNS resolved-IP recheck, the OWASP resolve-and-validate guidance, the RFC 1918 / RFC 3927 ranges, and the modelled local-stub verdicts captured under `qa-results/security/dns_rebinding/`) are grounded in the cited sources + the test's captured evidence. Items explicitly marked **INFERENCE** (§11.4.6) — the Squid behind-vs-beside placement, the reuse-vs-extend verdict, and the exact config mapping to `HELIX_BRIDGE_SUBNET` — are prototype-time determinations to be proven before being asserted as settled, never claimed here as decided. Deep-research (§11.4.150), multi-angle: mechanism (OWASP/DNS-rebinding), tool (smokescreen README + Fly.io operational write-up), standards (RFC 1918/3927), integration (our Squid/Dante floor).*